PAS LE PC MAISON · PROXIES HTTP TOURNANTS · DISQUE VPS D'ABORD

Architecture stream YouTube — comment c'est géré exactement

Le téléphone ne parle jamais à googlevideo.com avec son IP. Il demande `GET https://music.hubera.cloud/api/stream/<videoId>` (Bearer ou `?access_token=`). C'est le **conteneur Docker `ytmusic` sur le VPS** qui résout YouTube, passe par des **proxies HTTP CONNECT**, et pipe l'audio. Ton PC de salon n'est **pas** dans le chemin live.

> Health live au moment du rapport : `streamUpstream: null`, `upstreamAllowed: false`, `serverOauth: true`, `YOUTUBE_HTTP_PROXY_FREE` ON, cookies header-file configurés, pool free ~500–600 (post-restart ~30 le temps de recharger), `gvWinners` souvent 0 ou 1 (le vrai goulot froid).

Réponse directe à « on passe par des proxy, pas le PC ? »

Élément	État live
PC maison / tunnel 172.17.0.1:18788	**Hors chemin. Fichier stream-upstream.url supprimé. ALLOW_STREAM_UPSTREAM uniquement = 1|true (Portainer). Absent → resolveStreamUpstream() = null.**
Pourquoi c'était dangereux	Le fichier seul suffisait à « allumer » le relais. Docker gateway TCP ouvert, réponse vide → Exo BUFFERING infini même sur titres disque.
Proxies	**HTTP(S) CONNECT depuis le VPS vers des IPs publiques. Pas de SOCKS maison. Pas de trafic YouTube via ton IP résidentielle actuelle.**
OAuth TV	Comptes ytm_accounts sur volume ytmusic_data. Innertube TV + yt-dlp.

```
Téléphone / web
    │  GET /api/stream/:id  (+ Range)
    ▼
VPS Docker ytmusic  (tsx api/src/index.ts)
    │  1. upload user / tête RAM / disque .m4a OU .mp4 muxé  → 206 immédiat
    │  2. (opt-in) relais maison STREAM_UPSTREAM — OFF en prod
    │  3. getAudioFormat (Innertube OAuth TV / cookies / yt-dlp -g) VIA PROXY
    │  4. fetch googlevideo VIA LE MÊME PROXY (sinon 403 IP mismatch)
    │  5. pipe bytes → client   jamais 302 googlevideo vers le téléphone
    ▼
Proxies HTTP free pool (youtubeProxy.ts)  +  CDN googlevideo.com
```

Chemin d'un GET, dans l'ordre réel du code

1. **Auth.** 401 sans compte (sauf relais interne x-ytm-stream-relay).
2. **Upload perso.** MP3 importé sur le compte → pipe audio/mpeg, header X-PLM-Stream-Cache=user-upload.
3. **Tête RAM.** 1er Mo déjà vu (streamHeadCache, max 64 titres) → 206 sans disque ni YouTube.
4. **Disque early (hotfix 6 oct).** `playableAudioPath(id)` : `data/cache/<id>.m4a` s'il est assez gros et pas DASH, sinon `data/cache/<id>.mp4` muxé (clip YouTube itag 18/22). Servi tout de suite en `audio/mp4`. ffmpeg extract en fond, *jamais awaited* sur le GET (c'était le BUFFERING des titres déjà écoutés : Flowers n'avait que le mp4 3,2 Mo).
5. **Atlas / remplacement.** Autre videoId du même morceau déjà en cache → 302 interne vers cet id.
6. **Android froid.** Si rien sur disque : `downloadTrack` prefix progressif + micro-wait 350–700 ms. Plus de next-disk ×40 ni POST /api/download ×12 pins (ça mettait yt-dlp en cooldown).
7. **Range open-ended Android.** Forcé en `bytes=0-524287` pour un 1er octet rapide. Mid-range suivant sert le disque qui grossit.
8. **Relais maison.** Uniquement si ALLOW_STREAM_UPSTREAM. First-byte court puis fallback VPS. Live : sauté.
9. **Format + googlevideo.** Course disque vs getAudioFormat. Si format.viaProxy : fetch CONNECT, **pas** de 302 vers le phone (wantsDirectRedirect seulement ?redirect=1).
10. **403/401/404 GV.** Invalidate cache format, rotate proxy, yt-dlp -g only, reject DASH (ftyp=dash) → progressif.
11. **Warm.** POST /api/stream/warm : têtes biblio en prefix, next-disk cap 4, pas 24.

ISCOMPLETEENOUGHDISK

Taille ≥ 512 KiB et pas DASH → jouable, même si un ancien Content-Length googlevideo (souvent trop grand) avait « empoisonné » le 85 %. En-dessous, 96 KiB mini, et 85 % de l'annoncé si connu.

CLIENTS

Client	URL	Auth
Android Exo	/api/stream/id?access_token=… (+ Range)	query + header Exo DataSource
Prefetch	POST /api/stream/warm + GET ?warm=1	Authorization
Web	même API, proxy CORS ; pas de 302 GV	cookie / bearer
Hors-ligne phone	fichier local SimpleCache / .m4a	—

Pool de proxies — rotation, score, stripes

Fichier d'en-tête du module : « Proxies HTTP pour yt-dlp **et** le relais googlevideo. Contourne les 5xx / LOGIN_REQUIRED de l'IP datacenter VPS — pas le PC maison. »

SOURCES, DANS L'ORDRE

1. `YOUTUBE_HTTP_PROXY` — un proxy fixe (optionnel, pas en tête des pools user pour ne pas brûler une IP).
2. `YOUTUBE_HTTP_PROXY_LIST` — CSV / fichier, une URL par ligne.
3. `YOUTUBE_HTTP_PROXY_FREE=1` (défaut VPS prod) — listes publiques, cap 600, TTL 8 min, refresh fond 5 min.
4. Direct VPS en dernier si pool ON (`directLast`). Direct en premier seulement si pas de probe / pas de free.

INTELLIGENCE

Mécanisme	Détail
Probe TCP / HTTP CONNECT	1,2–2,4 s. Mort → deadUntil. Évite 12 s yt-dlp sur un cadavre.
Canary	GET https://www.google.com/generate_204 via le proxy.
Score GV	206/200 googlevideo = gvHits (winner). 5xx/403 = gvMiss. 3 miss sur un hot → quarantaine exponentielle 15 min × 2^n + jitter.
Éviction	4 fails + cooldown 4 min → drop. Pool recap à 600, on garde les gvHits.
Hot pool disque	data/hot_pool.json, 15–30 proxys GV récents, expire 6 h. Rotation : YouTube bloque une IP unique.
Lease user	24 proxys / user (env YOUTUBE_PROXY_USER_POOL 8–48). Lecture et DL ne se marchent pas dessus.
Stripe	hash32(url)&1 → purpose stream | download. Pénurie → recoupe l'autre stripe.
CONNECT allowlist	youtube / googlevideo / google generate_204 seulement (anti open-relay).
Keep-alive	https.Agent par proxy, CONNECT+TLS réutilisé vers le même googlevideo (même IP que le -g).
Course Gen2	raceGen2Proxies : 2–3 Innertube en parallèle, 1er URL GV gagne, timeout 8 s. Plus de série 8 s × 3.

LIAISON IP OBLIGATOIRE

L'URL googlevideo est signée pour **l'IP du proxy qui a fait Innertube / yt-dlp -g**. Si le VPS fetch ensuite en direct, ou si on 302 le téléphone : 403. D'où `format.viaProxy` + `fetchUrlViaProxy` + pas de redirect Exo.

> Goulot actuel : `gvWinners` souvent 0–1. 537 proxys « TCP ok » ≠ 537 proxys qui parlent à googlevideo. Les listes free sont du datacenter sale. Un titre froid part alors en roulette 5–40 s.

Résolution d’URL audio (avant le pipe)

1. Cache format mémoire (décryptage Player JS déjà fait).
2. Innertube youtubei.js, client TV OAuth (ytm_accounts volume) — itag 140 AAC ami Exo.
3. Cookies navigateur optionnels (header-file). Health : « pas de Premium requis ».
4. yt-dlp binaire local `bin/yt-dlp` : `-g` (URL seule) ou download progressif itag 140 / 18. Args cookies + extractor + `ytDlpProxyCliArgs(proxy)`.
5. Gate concurrence yt-dlp (slots live vs idle). Cooldown bot « Sign in to confirm » par user → 503 retry later, pas une boucle Exo.

CE QU’ON MET SUR DISQUE

Fichier	Rôle
<id>.m4a	Audio progressif (préféré Exo / offline).
<id>.mp4	Clip muxé (onglet Vidéo + fallback lecture titre depuis le 6 oct).
ftyp=dash	Purgé. Exo et l’offline mobile le refusent.

Volume ~23 Go. ~4942 m4a + 377 mp4. Les mp4 sans m4a étaient injouables en « mode titre » avant le hotfix (ffmpeg extract souvent KO : stream id [0][0][0][0] — on sert le mp4 tel quel, Exo lit l’AAC).

WARM / FILE D’ATTENTE

- Têtes biblio : prefix only (enqueueListHeadWarm), plus de enqueueNextDiskWarm ×40.
- Next-disk : cap 4 hot / 8 idle. POST /api/download : prefix si déjà chaud, sinon enqueue borné, pas d’await yt-dlp.
- Android ne fait plus requestServerDiskCache ×12 pins au Home.
- Pendant une lecture : suspendBackgroundDiskWarm garde courant+3, vide la file search.

D’où vient le délai (et ce qui est déjà < 2 s)

Cas	Chemin	TTFB typique
Titre .m4a complet VPS	early disk 206	**< 150 ms**
Titre .mp4 muxé seulement	early disk mp4 (hotfix)	**< 200 ms (était infini)**
Tête RAM	1er Mo	**< 80 ms**
Prefix en cours de DL	disk-growing Range	**0,3–2 s si le fichier ≥ 64–512 KiB**
Froid, format encore en cache + proxy GV chaud	fetchGV viaProxy	**0,4–2 s**
Froid, Innertube + 0 gvWinner	roulette proxies + yt-dlp	**5–40 s, parfois 502**
Relais maison mort (avant fix)	TCP open, body vide	**BUFFERING infini pos=0**
await ffmpeg sur GET (avant fix)	extractAudioFromMuxed	**blocage titres chauds muxés**

Objectif « n’importe quelle musique YouTube < 2 s »

Impossible à garantir à 100 % avec des proxies gratuits + IP VPS datacenter. YouTube classe l’IP, lie l’URL à cette IP, et rate-limit. En revanche on peut viser **< 2 s sur ≥ 95 % des plays** : tout ce qui a déjà été entendu une fois (disque), plus les froids si on a 15–30 IPs résidentielles stables.

Solutions pour rester autour de < 2 s

A. DÉJÀ EN PLACE — À GARDER

- Disque / RAM d'abord, mp4 muxé jouable, pas d'await ffmpeg, pas de relais maison fantôme.
- Prefix 512 KiB pendant le play du courant (warm +1 / +2 / +3).
- Course 2–3 proxys Gen2, stripe stream≠download, 302 GV interdit vers le phone.
- OAuth TV VPS + cookies optionnels.

B. GAINS RAPIDES (RECOMMANDÉ MAINTENANT)

#	Solution	Effet < 2 s	Coût / risque
B1	**10–30 proxies résidentiels payants** (Bright Data, Oxylabs, IPRoyal residential sticky 10 min) en YOUTUBE_HTTP_PROXY_LIST, free en appoint	gvWinners réels ; froid souvent 0,5–1,5 s	20–80 €/mois ; sticky session obligatoire (même IP Innertube+GV)
B2	Sonde GV Range 0-1 en continu sur le hot pool (déjà coded probe) — ne mettre en stripe que les gvHits>0	Évite 8 s de TCP-ok / GV-ko	Déjà dans youtubeProxy ; forcer pickHotProxies en tête de race
B3	Après 1er play : garder le .m4a (ou extraire mp4 en idle, pas sur GET) + CDN interne (même volume)	2e écoute toujours < 200 ms	Disque ; déjà 23 Go
B4	1 VPS / LXC IP résidentielle (4G/box) *uniquement* pour Innertube+GV, pas le PC allumé/éteint	Comme l'ancien relais mais 24/7	Un mini-host toujours on ; ALLOW_STREAM_UPSTREAM seulement si health 200

C. ARCHITECTURE « PRESQUE TOUJOURS < 2 S » (CIBLE)

1. **Edge audio Hubera** : premier GET froid → résolution proxy résidentiel (B1) → écriture .m4a → les GET suivants (Range, skip, autre téléphone du même compte) = disque VPS. Objectif : 1 seul round YouTube par titre et par semaine.
2. **Prefetch prédictif borné** : uniquement courant+3, prefix 512 KiB, jamais 12 pins complets (déjà le cas 1.3.352).
3. **Cache partagé compte** : rememberReadyAudio / atlas (déjà) pour que Samsung joue le m4a téléchargé par le Nothing.
4. **File hors-ligne téléphone** : SimpleCache têtes ; hors-ligne = fichier local (règle mesh).

D. PISTES À NE PAS PRENDRE

- Rallumer le tunnel 172.17.0.1:18788 / PC éteint → BUFFERING vide.
- 302 googlevideo vers Exo → 403 IP.
- YouTube Premium « officiel » côté client : l'app n'en a pas besoin ; ça n'aide pas l'IP VPS.
- Multiplier yt-dlp parallèle sur toute la biblio → cooldown bot, tout le monde à 0.
- Invidious / Piped publics : mêmes 403, SLA pire, dépendance tierce.

Autres leviers (moyen / long terme)

#	Idée	Pour < 2 s	Commentaire
E	Object storage (R2/S3) des .m4a après 1er play, CDN Cloudflare devant music.hubera.cloud/stream-static/	Oui pour le 2e hit mondial	Droit d'auteur : rester privé compte, pas un YouTube public
F	Mesh LAN entre téléphones (titre déjà dans SimpleCache du Samsung → Nothing)	Oui hors-ligne / même Wi-Fi	Aligné règle mesh labo ; plus tard
G	Player clients multiples (ANDROID, IOS, TV, WEB) en hedge plus large	Plus de chances d'URL	Déjà TV + yt-dlp ; ne pas exploser les slots
H	IPv6 dual + plusieurs VPS EU (OVH + Hetzner) chacun son pool	Moins de ban unique	Complexité JWT/volume unique

Reco concrète pour la semaine : (1) garder le hotfix disque mp4 — déjà live ; (2) coller 10 sticky residential dans YOUTUBE_HTTP_PROXY_LIST et mesurer gvWinners ; (3) extraire les 377 mp4 → m4a en job idle (ffmpeg -vn) pour ne plus dépendre du conteneur MP4 ; (4) ne jamais rerouter le live via le PC.

Fichiers de référence

- `products/music/api/src/media/stream.ts` — GET stream, disque, maison, pipe.
- `products/music/api/src/youtube/youtubeProxy.ts` — pool, stripe, CONNECT, race.
- `products/music/api/src/youtube/yt.ts` — Innertube / yt-dlp / cache URL.
- `products/music/api/src/youtube/youtubeCookies.ts` — cookies + args yt-dlp.
- `products/music/api/src/index.ts` — /api/version, /api/stream, /api/health.
- Android `AppContainer.streamUrl`, `PlaybackService`, `StreamPrefetcher`.

Santé à relire

`GET https://music.hubera.cloud/api/health` → playback.streamHeadCache, streamUpstream, streamAuth.youtubeProxy.{poolSize,usable,gvWinners,hotPool,leased}.

Client N-1 : `GET /api/version?clientVersion=1.3.351&clientVersionCode=10651&clientPackage=cloud.hubera.music` → apkAvailable.